```
- - -
- hosts: webservers
  gather_facts: yes
  remote_user: <sudo User>
  sudo: yes

  tasks:
    - name: transfer and run Ubuntu specific script
      script: provision_at_ubuntu.sh arg1 arg2
      register: provdeb
      when: ansible_os_family == "Debian"
    - debug: var=provdeb.stdout_lines

    - name: transfer and run RHEL specific script
      script: provision_at_centos.sh arg1 arg2
      register: provrhel
      when: ansible_os_family == "Redhat"
    - debug: var="provrhel.stdout_lines"

    - hosts: dbservers
      gather_facts: no
      remote_user: <sudo User>
      sudo: yes

  tasks:
   - name: transfer and run diagnostic script one host at a
time
     script: dump_diagnostics.sh arg1 arg2
     register: dumpdiag
     serial: 1
   - debug: var=dumpdiag.stdout_lines
```

This playbook demonstrates some useful constructs provided by Ansible. The facts gathered by Ansible are useful to run your task based upon some characteristics of the remote nodes like type of operating system, etc. The *register* keyword assigns the output of a task to some variable, and *debug* is a module that is useful to dump a variable. You can use a serial keyword to define a batch of hosts on which a task is executed in the serial manner. There is a looping functionality provided by the *with_items* keyword that runs a task over a list of items. Ansible also provides a module called *raw*, which enables you to run a command over SSH with the need for Python on remote hosts. This is helpful to bootstrap the necessary components like Python, etc, on 'Pythonless' hosts like routers or other devices, and then use the full module system provided by Ansible.

The modules *shell* and *command* are other ways to run remote commands in the playbooks. The main difference between these modules is that the *shell* module runs the command through */bin/sh* and understands variables like $HOME, "<", ">", "|" , "&", etc. Ansible stops a playbook run when it first encounters any error after running the command, but you can alter this behaviour by using *ignore_errors*, in case you just want to fire off some remote commands, irrespective of their return codes. The following example shows how easily you can trigger a set of commands on remote hosts:

```
- - -
    - hosts: redis
      gather_facts: no
      remote_user: <sudo User>
      sudo: yes

  tasks:
    - name: run commands
      shell: "{{ item }}"
      register: output
      with_items:
        - dmidecode > /tmp/biosdump.log 2>&1
        - cd /tmp && ifconfig > nwiface.log 2>&1
        - sync
        - df -kh

     - dump: var=output.stdout_lines
```

I have only touched on Ansible modules that execute remote commands and some basic features of the playbooks in this article. But Ansible is so feature rich and powerful that it could handle infrastructure comprising thousands of cloud or physical servers, single-handedly. The detailed exploration of Ansible involves a learning curve and could produce material. But you could get started with the knowledge gained in this article, and go through the Ansible documentation to dig deeper into the subject and create complicated workflows with it. END

References

[1] sshpass homepage: *http://sourceforge.net/projects/sshpass/?source=navbar*

[2] Fabric homepage: *http://www.fabfile.org*

[3] Python for Windows: *https://www.python.org/downloads/windows/*

[4] pip for Windows: *https://sites.google.com/site/pydatalog/python/pip-for-windows*

[5] pycrypto for Windows: *http://www.voidspace.org.uk/python/modules.shtml#pycrypto*

[6] Ansible config file: *https://raw.githubusercontent.com/ansible/ansible/devel/examples/ansible.cfg*

[7] Ansible documentation: *http://docs.ansible.com*

By: Ankur Kumar

The author is a systems and infrastructure developer/architect and researcher.